

Using extreme value theory for VaR calculation

In this section, we are going to implement extreme value theory (EVT) in order to calculate VaR. The following Python code can be used to calculate VaR for both the delta normal and the historical distribution on a daily basis using S&P500 data. The data can be retrieved from the Excel workfile 'sp500_daily' and stored in the Python DataFrame **data**, details of which can be found in the first command block. After computing the daily log returns of the S&P500 index, we obtain the parameters (i.e., the mean and standard deviation) later used to describe the distribution. The parameters can easily be calculated by the commands **mean = np.mean(data['ret'])** and **std_dev = np.std(data['ret'])**.

```
In [1]: import Pandas as pd
import NumPy as np
import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
from scipy.stats import norm

abspath = 'C:/Users/tao24/OneDrive - University of Reading/PhD/' \
          'QMF Book/book Ran/data files new/Book4e_data/'
data = pd.read_excel(abspath + 'sp500_daily.xlsx', index_col=[0])

def LogDiff(x):
    x_diff = np.log(x/x.shift(1))
    x_diff = x_diff.dropna()
    return x_diff

data['ret'] = LogDiff(data['Close'])
data = data.rename(columns={'Close': 'sp500'})
data = data.dropna()

mean = np.mean(data['ret'])
std_dev = np.std(data['ret'])
```

Next, we plot both a histogram and the probability density function (pdf) of the standard normal distribution given the calculated parameters (**mean**, **std_dev**). As can be seen in the output window (Figure 1), both distributions are very close to each other given that the input sample size is quite large.

```
In [10]: fig = plt.figure(1, dpi=600)
ax1 = fig.add_subplot(111)
ax1.hist(data['ret'], 100, edgecolor='black', linewidth=1.2)
ax1.set_ylabel('density')

x = np.linspace(mean - 5*std_dev, mean + 5*std_dev, 100)
ax2 = ax1.twinx()
ax2.plot(x, mlab.normpdf(x, mean, std_dev), "r")
ax2.set_ylabel('pdf of the normal distribution')
plt.legend()
plt.show()
```

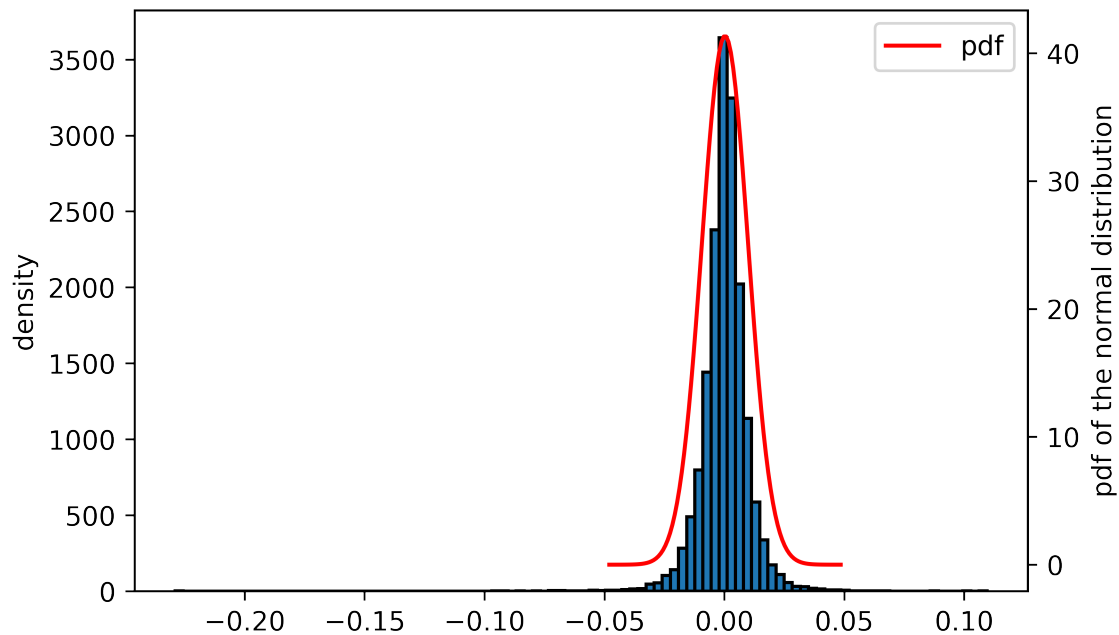


Figure 1: Histogram and the Probability Density Function of the Standard Normal Distribution

There are several ways to compute value at risk. The first calculation assumes that the return distribution is normal, and then we can simply take the critical value from the normal distribution at the α significance level multiplied by the standard deviation σ of the sample and subtracting the mean value.

$$VaR_{normal} = \sigma * Z_{\alpha} - \mu \quad (1)$$

In Python, the calculation can easily be done by applying the **ppf** function (i.e., the Percent Point function, which is the inverse of cdf – percentiles) from the **scipy.stats.norm** module. The specific commands are provided as follows. For VaR at the 90% confidence level, the input quantile is simply 1-0.9. A 95% VaR would be 1-0.95, and so on.

```
In [11]: # normal distribution
          VaR_90 = norm.ppf(1-0.9, mean, std_dev)
          VaR_95 = norm.ppf(1-0.95, mean, std_dev)
          VaR_99 = norm.ppf(1-0.99, mean, std_dev)

          print('90%:{}'.format(VaR_90) )
          print('95%:{}'.format(VaR_95) )
          print('99%:{}'.format(VaR_99) )
```

```
90%:-0.012067156368399046
95%:-0.01557235218232069
99%:-0.022147516586450203
```

Alternatively, VaR can be calculated by sorting the portfolio returns and then simply selecting the quantile from the empirical distribution of ordered historical returns. In this setting, the non-parametric VaR estimation would be under no assumptions, which is therefore fairly simple. Specifically, the Pandas module provides the quantile function where the corresponding VaR can be obtained by finding the appropriate quantiles, as shown in the following commands.

```
In [12]: # historical distribution
        VaR_90 = data['ret'].quantile(0.1)
        VaR_95 = data['ret'].quantile(0.05)
        VaR_99 = data['ret'].quantile(0.01)

        print('90%:{}'.format(VaR_90) )
        print('95%:{}'.format(VaR_95) )
        print('99%:{}'.format(VaR_99) )
```

```
90%:-0.009886539705571379
95%:-0.014438124291662094
99%:-0.025965714775369006
```

Finally, let us spend a few minutes comparing the VaR calculated by the different methods. Clearly, the historical 99% VaR is larger in absolute value than the corresponding normal VaR (−0.026 vs −0.022). The rationale behind this finding is that real financial data tend to have a distribution with fatter tails than the assumed standard normal distribution.

The Hill estimator for extreme value theory

In this subsection we will examine the tail of the distribution to estimate the VaR by implementing the Hill estimator of the shape parameter ζ of an extreme value distribution. The code to achieve this is presented as follows.

```
In [5]: threshold = -0.025
        alpha = 0.01

        data1 = data[data['ret']<=threshold]
        data1 = data1.sort_values(['ret'])
        data1['k'] = data1.iloc[0]['ret']

        xi = (np.log(-data1['k']) - np.log(-data1['ret'])) .sum()*(1/(data1.shape[0]-1))
        VaR_hill = data1['ret']*(data.shape[0]/data1.shape[0]*alpha)**(-xi)
        VaR_hill = VaR_hill[1:]
```

The threshold is set to be −0.025, which determines the number of observations identified as being in the tail. Note that this value is subjective and readers can choose an appropriate threshold at their own discretion. α is equal to 0.01 and corresponds to a 99% confidence level. To calculate the tail-index ζ using the Hill estimator, we order the data over the threshold from the largest value (lowest log returns) to the smallest value (largest log returns). By following the Hill estimator formula below, we then obtain the ζ and calculate the VaR .

$$\hat{\zeta} = \frac{1}{k-1} \sum_{i=1}^{k-1} [\ln(\tilde{y}_i) - \ln(\tilde{y}_k)] \quad (2)$$

```
In [6]: plt.figure(2, dpi=600)
        x = [i for i in range(1, VaR_hill.shape[0]+1)]
        plt.scatter(x, VaR_hill)
        plt.xlabel('n')
        plt.ylabel('VaR')
        plt.show()
```

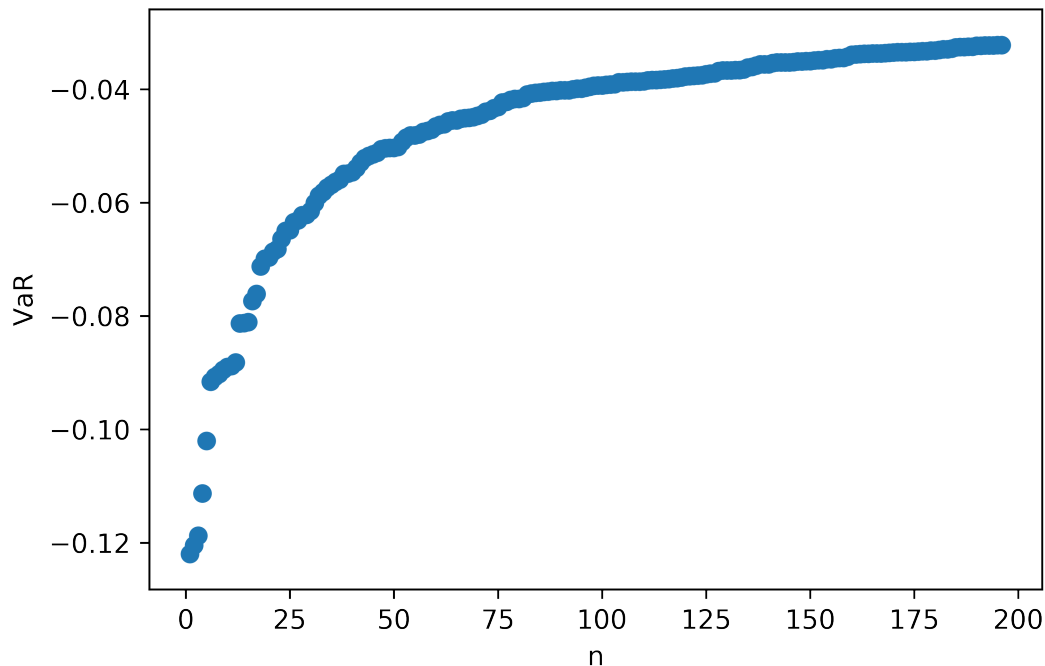


Figure 2: Hill Plot

Looking at the Hill plot (see Figure 2), we can see that the estimated VaR starts from a very negative value (-0.12) and gradually converges on -0.032 , which suggests that the expected maximum is 3.2%

Parameter estimation for extreme value theory

The three parameters ξ , μ , σ used to define an extreme value distribution can be estimated by maximum likelihood. To do so, we need to set up a log-likelihood function with a certain distribution and find the parameter values to maximise it. The Python code below demonstrates this in detail. To be consistent with the previous section, the same values are set up for both alpha and the threshold. All observations exceeding the threshold $\tilde{y}_t$ are obtained and stored in the variable **rets**. Next, we set up initial values for the distribution's parameter ξ and σ and we allow them to be optimised in the later maximum likelihood estimation.

```
In [7]: import NumPy as np
        from scipy.optimize import minimize

        alpha = 0.01
```

```

threshold = -0.025
y = threshold - data[data['ret']<=threshold]
y = y.sort_values(['ret'])
rets = y['ret'].values

shape = 0.2
scale = 1
x0 = (shape, scale)

```

We then define a custom function where the natural log of the probability density function is added up over the tail observations. By minimising the log-likelihood function by the function **minimize** from the module **scipy.optimize**, we are able to obtain optimised parameters, which are 0.38803644 and 0.0075239 for the distribution's shape and scale in this case.*

```

In [8]: def fun(params,rets):
        F = lambda x: 1/params[1]*(1 + params[0] * x/params[1])**(-1 - 1/params[0])
        result = -sum([np.log(d) for d in [F(x) for x in rets] ])
        return result

        result = minimize(fun, x0, args=rets, method='Nelder-Mead')
        print (result)

final_simplex: (array([[ 0.38803644,  0.0075239 ],
 [ 0.38811469,  0.00752272],
 [ 0.38812762,  0.00752404]]), array([-689.8224412 , -689.82244075, -689.82244058]))
        fun: -689.82244119932989
        message: 'Optimization terminated successfully.'
        nfev: 99
        nit: 51
        status: 0
        success: True
        x: array([ 0.38803644,  0.0075239 ])

```

Finally, we calculate the VaR under maximum likelihood, which suggests the expected maximum loss is 2.5%.

```

In [9]: shape = result.x[0]
        scale = result.x[1]

        VaR_mle = threshold + scale/shape*((data.shape[0]/y.shape[0]*alpha)**(shape)-1)
        print('VaR_mle:{}'.format(VaR_mle) )

```

```

VaR_mle:-0.025975570332890704

```

*Note that, effectively, we are minimising the negative of the log-likelihood function, which is exactly equivalent to maximising the log-likelihood function but is simpler to implement since it means that we are able to use the **minimize** function in Python.